

Table des matières

Mini-cas — La Brûlerie, un mois de caisse de bout en bout.....	1
Étape 0 — Le café, et ce que vous allez bâtir	1
Étape 1 — Le projet, à côté de celui de Boréal	2
Étape 2 — Le brut figé, chargé par dbt seed	3
Étape 3 — L'étoile, sur papier	5
Étape 4 — Le STAGING, quatre vues minces.....	6
Étape 5 — Les MARTS, une étoile.....	7
Étape 6 — La preuve : les tests et le témoin.....	8
Étape 7 — Documenter, et répondre	9
Récapitulatif	10
Annexe — L'arborescence et les commandes	11

Mini-cas — La Brûlerie, un mois de caisse de bout en bout

Mini-cas figé — Formation Analytics Engineering · les démarches des Cours 01 à 07, en petit

Étape 0 — Le café, et ce que vous allez bâtir

La Brûlerie du Passage est un café de quartier à Montréal. Elle torréfie son café sur place, sert au comptoir et n'a qu'une caisse. Le mini-cas en prend un mois, juin 2025, figé en fichiers dans `Data_Domain/_univers/brulerie/mini_cas/sources/` : il n'y a rien à générer, les données sont versionnées avec le parcours.

Vous allez refaire seul, sur ce mois, l'arc que Boréal vous a fait parcourir pièce par pièce :

les CSV de graines/ → RAW, chargée par dbt seed → STAGING, quatre vues → MARTS, une étoile testée et documentée.

Ce document ne réexplique rien. Chaque étape nomme la section de cours qui fonde son geste, et ne garde que ce qui est propre à ce cas. Trois choses le sont : le brut entre par dbt seed et non par une source ; le cas est assez petit pour qu'un ticket se suive de tête, du CSV jusqu'au fait ; et deux décisions de contenu, les tickets annulés et les taxes, que Boréal ne pose pas.

Il se fait une fois le Cours 07 derrière vous, parce que c'est lui qui charge le premier seed, et avant la synthèse Aurore. Comptez une séance, deux heures environ : 55 minutes de lecture, mise en situation comprise, et un peu plus d'une heure pour écrire le projet, dont une bonne part sans bloc à recopier.

Le dossier graines/ porte le mois en quatre fichiers, prêts pour un seed. Le sens de chaque colonne est dans sources/dictionnaire_brulerie.md.

Fichier	Lignes	Ce qu'il porte
tickets.csv	1 527	un ticket par ligne : type, statut, membre, sous-total, taxes, encaissé
lignes.csv	2 532	un produit vendu par ligne : quantité, prix, rabais, montant
produits.csv	32	le catalogue au 2025-06-30
membres.csv	994	le programme de fidélité au 2025-06-30

Le corrigé est dans Solution/02_ae/, le projet dbt tel qu'il doit être à la fin. Chaque bloc de ce document qui appartient au projet en est tiré, sans ses commentaires d'en-tête.

Étape 1 — Le projet, à côté de celui de Boréal

Les gestes du Labo 03, étapes 1 à 3, avec d'autres noms. La théorie est au Cours 03 : le projet (Theorie_03.md, section 2), la connexion (section 3) et le schéma où dbt écrit (section 5).

Le mini-cas vit dans son propre dossier, Data_Domain/Projet_Brulerie/, que Git ignore comme Projet_Boreal/. Le projet dbt en occupe un sous-dossier, 02_ae, qui est aussi un dépôt ; la base DuckDB se range à côté, hors du dépôt. Depuis l'Anaconda Prompt, env formations actif :

```
cd Data_Domain
mkdir Projet_Brulerie\02_ae
cd Projet_Brulerie\02_ae
git init
```

Le profiles.yml que vous avez écrit au Cours 03 reçoit un second profil, à la suite du premier. Un seul fichier, un bloc par projet, et la ligne profile: de chaque dbt_project.yml dit lequel lire :

```
brulerie_ae:
  target: dev
  outputs:
    dev:
      type: duckdb
      path: C:/Users/<vous>/.../Data_Domain/Projet_Brulerie/mini_cas.duckdb
      schema: staging
      threads: 4
```

Le reste se recopie de Boréal. Le `.gitignore` est le même, et le `packages.yml` déclare le même `dbt_utils`, pour `generate_surrogate_key` et `date_spine`. La macro `generate_schema_name` est celle du Labo 03, étape 3, et non celle du Cours 07 : un mois figé n'a pas de production à séparer du développement. Le `dbt_project.yml`, pour commencer :

```
name: brulerie_ae
version: 1.0.0
profile: brulerie_ae

model-paths: ["models"]
macro-paths: ["macros"]
test-paths: ["tests"]
seed-paths: ["seeds"]
target-path: "target"
clean-targets: ["target", "dbt_packages"]

seeds:
  brulerie_ae:
    +schema: raw

models:
  brulerie_ae:
    staging:
      +materialized: view
      +schema: staging
    marts:
      +materialized: table
      +schema: marts
```

Puis `dbt deps` et `dbt debug`, qui doit rendre « All checks passed! ».

Étape 2 — Le brut figé, chargé par `dbt seed`

Le seed est au Cours 07 (Theorie_07.md, section 8), et la règle de `ref()` et `source()` au Cours 03 (Theorie_03.md, sections 4 et 9).

Copiez les quatre CSV dans le dossier `seeds/` du projet, puis chargez-les :

```
mkdir seeds
copy ../../_univers\brulerie\mini_cas\sources\graines\*.csv seeds\
dbt seed
```

Attendu : quatre tables dans le schéma `raw`, 1 527 tickets, 2 532 lignes, 32 produits et 994 membres.

2.1 Un seed s'appelle par ref()

La règle du Cours 03 s'applique à la lettre : `source()` pour ce qui vient d'ailleurs, `ref()` pour tout ce que le projet fabrique. Le contenu de ces tables vient de la caisse, mais c'est `dbt seed` qui les a créées, donc elles s'appellent par `ref('tickets')`, comme un modèle. Le projet n'a donc aucun `_sources.yml`, et pas de fraîcheur à surveiller : un mois figé n'en a pas.

Le Cours 07 disait qu'un seed est une bonne réponse à cinq lignes et une mauvaise à cinq mille. Avec 2 532 lignes, le mini-cas est à la limite, et il la tient pour une seule raison : ces fichiers ne changeront jamais. Une caisse qui exporte chaque soir serait une source, chargée hors de `dbt`.

2.2 Ce que `dbt seed` devine

Un CSV n'a pas de types, et `dbt seed` en devine. Regardez ce qu'il a choisi, dans la ligne de commande DuckDB lancée depuis `Projet_Brulerie/`, et fermez-la avant de relancer `dbt` : un seul processus à la fois sur la base, comme chez Boréal.

```
duckdb mini_cas.duckdb

select column_name, data_type
from information_schema.columns
where table_schema = 'raw' and table_name = 'tickets';
```

Les entiers, les dates et l'horodatage sont justes. Les dix colonnes de montants des trois fichiers sont en `DOUBLE`, un nombre à virgule flottante, et c'est un défaut qui ne se voit qu'au total :

```
select sum(encaisse), sum(tvq), sum(tvq) = 1295.66
from raw.tickets
where statut = 'PAYE';
```

Le mois rend 18666.380000000012 d'encaissé au lieu de 18 666,38 \$, et une TVQ de 1295.6599999999978 que la comparaison avec 1 295,66 déclare fausse, pour moins d'un milliardième de cent. Ticket par ticket, rien ne dépasse ; c'est en additionnant quinze cents tickets que l'erreur d'arrondi du binaire s'accumule.

Chez Boréal, la question ne se posait pas : la RAW arrive toute en texte, et le typage se fait au `STAGING`. Ici, c'est vous qui chargez, donc le type se déclare au chargement, dans le bloc `seeds` : du `dbt_project.yml` :

```
seeds:
  brulerie_ae:
    +schema: raw
    tickets:
      +column_types:
        sous_total: decimal(10, 2)
        tps: decimal(10, 2)
        tvq: decimal(10, 2)
        total: decimal(10, 2)
        arrondi: decimal(10, 2)
        encaisse: decimal(10, 2)
```

```
lignes:
+column_types:
  prix_unitaire: decimal(10, 2)
  rabais: decimal(10, 2)
  montant: decimal(10, 2)
produits:
+column_types:
  prix: decimal(10, 2)
```

Piège — un `dbt seed` simple ne change pas le type d’une table déjà chargée. Il vide la table et la remplit, sans la recréer, et rend quand même `PASS=4` : mesuré, les montants restent en `DOUBLE` et la requête de contrôle rend les mêmes chiffres. Il faut `dbt seed --full-refresh`, qui recrée les tables. La requête rend alors 18666.38 et `true`.

Étape 3 — L’étoile, sur papier

La recette en quatre temps est au Cours 02 (`Theorie_02.md`, sections 2 à 4), et la dimension dégénérée à la section 8.

Décision	Pour La Brûlerie
Le processus	la vente au comptoir
Le grain	une ligne d’un ticket payé
Les dimensions	la date, le produit, le membre du programme de fidélité
Les mesures	la quantité, le montant brut, le rabais, le montant
La dimension dégénérée	le numéro du ticket, qui reste dans le fait

Deux décisions de contenu restent, et c’est le fait qui les prend, jamais le `STAGING`.

3.1 Ce qui est une vente

Le mois compte 15 tickets annulés. Ils gardent leurs 23 lignes et leurs 136,00 \$ de montants, mais rien n’a été encaissé : ils sortent du fait. Le mois compte aussi un remboursement : une chocolatine de 4,00 \$ vendue le 25 juin, rendue le 27. Il reste, en négatif, parce que les deux lignes disent ce qui s’est passé. Le fait porte donc 2 509 lignes pour 16 718,00 \$, exactement la somme des sous-totaux que la caisse a imprimés sur ses tickets payés.

3.2 Les taxes restent au grain du ticket

La TPS et la TVQ (la taxe fédérale et la taxe québécoise sur les ventes) se calculent sur les lignes taxables du ticket, puis s’arrondissent au cent une seule fois. Le ticket 158414, le deuxième du mois, se refait de tête :

Ligne	Produit	Taxe	Montant	TPS de la ligne, 5 %
1	Pérou Cajamarca, un sac de 340 g	détaxé	21,50 \$	aucune
2	Thé infusé, deux	taxable	7,50 \$	0,375 \$, arrondie à 0,38 \$
3	Latte	taxable	5,75 \$	0,2875 \$, arrondie à 0,29 \$

La caisse calcule 5 % de 13,25 \$, soit 0,6625 \$, et imprime 0,66 \$. Répartie puis arrondie ligne par ligne, la même taxe donne 0,67 \$. Sur le mois, la répartition s'écarte de la caisse sur 337 des 1 464 tickets qui ont une ligne taxable, et le mois compte 0,75 \$ de TPS de trop et 2,30 \$ de TVQ de moins que ce que la caisse a perçu.

Aucune règle de répartition ne retombe sur les montants imprimés, donc les taxes ne descendent pas au grain de la ligne. Elles restent dans `stg_tickets`, au grain du ticket. Une analyse qui en a besoin demande un second fait, au grain du ticket, pas une répartition.

Étape 4 — Le STAGING, quatre vues minces

Les quatre gestes sont au Cours 03 (`Theorie_03.md`, sections 6 et 7), et ce qu'une dimension de personnes a le droit de porter au Cours 06 (`Theorie_06.md`, section 6).

Une vue par seed, `stg_tickets`, `stg_lignes`, `stg_produits` et `stg_membres`, qui lit son seed par `ref()`. Les quatre gestes de Boréal se réduisent ici presque à rien, et c'est normal :

Le geste	Ce qu'il devient chez La Brûlerie
Renommer	rien : les noms de la caisse sont déjà lisibles
Typer	les montants le sont depuis le chargement ; reste la date du jour, tirée de l'horodatage
Décoder	les drapeaux O et N du catalogue et de l'infolettre deviennent des booléens
Tirer l'état courant	déjà fait : <code>graines/</code> ne garde que le dernier export de chaque jour

Le dernier geste mérite un mot. Les exports bruts de la caisse, dans `sources/caisse/`, livrent le 12 juin deux fois : 58 tickets exportés à 15 h 42, puis 65 le lendemain matin. Qui chargerait ces fichiers-là compterait la journée deux fois, et aurait à tirer l'état courant comme au Cours 03, section 7. Les graines vous en dispensent, et c'est la seule faveur qu'elles vous font.

Aucune vue ne filtre : les annulés et le remboursement restent au STAGING. Une seule mérite d'être montrée, celle des membres, pour ce qu'elle ne prend pas :

```

select
  membre_id,
  left(code_postal, 3) as secteur_postal,
  inscrit_le::date as date_inscription,
  infolettre = '0' as consent_infolettre

from {{ ref('membres') }}

```

Ni le prénom, ni le nom, ni le courriel, ni le téléphone ne montent. Du code postal, seuls les trois premiers caractères, le secteur de tri postal, qui dit de quel coin de la ville vient un membre sans dire où il habite. C’est la règle du Cours 06, appliquée dès le STAGING comme chez Boréal.

Étape 5 — Les MARTS, une étoile

La clé de substitution est au Cours 04 (Theorie_04.md, section 3), dim_date à la section 4 et le fait à la section 7.

Trois dimensions. dim_produit et dim_membre reçoivent leur clé de generate_surrogate_key ; celle de dim_date est la date elle-même, comme chez Boréal.

Dimension	Lignes	Ce qu'elle porte
dim_date	30	celle de Boréal, bornée au mois : du 2025-06-01 au 2025-06-30, dont 9 jours de fin de semaine
dim_produit	32	le catalogue au 2025-06-30, en type 1 ; aucune hausse de prix ne tombe dans le mois, et le prix du catalogue égale celui des 2 532 lignes
dim_membre	994	le membre, son secteur, sa date d’inscription, son consentement à l’infolettre

Puis le fait, qui porte les deux décisions de l’étape 3 :

```

select
  t.date_vente as cle_date,
  {{ dbt_utils.generate_surrogate_key(['l.produit_id']) }} as cle_produit,
  case
    when t.membre_id is not null
      then {{ dbt_utils.generate_surrogate_key(['t.membre_id']) }}
  end as cle_membre,
  l.ticket_id,
  l.ligne,

```

```

l.quantite,
l.quantite * l.prix_unitaire as montant_brut,
l.rabais,
l.montant

from {{ ref('stg_lignes') }} as l
inner join {{ ref('stg_tickets') }} as t
  on t.ticket_id = l.ticket_id

where t.statut = 'PAYE'

```

Le case autour de la clé du membre répond au piège que le Cours 04 annonçait : la macro remplace une valeur vide par un texte sentinelle, donc deux clés vides reçoivent le même hachage. Ici, la clé vide n'est pas un défaut, c'est un client qui n'a pas présenté de carte, et 1 626 lignes du mois sont dans ce cas. Sans le case, elles portent toutes la même clé, qui ne désigne aucun membre. Avec lui, elles portent une clé nulle, et c'est ce qu'elles veulent dire.

Le ticket 158414 se retrouve dans le fait en trois lignes, 21,50 \$, 7,50 \$ et 5,75 \$, soit 34,75 \$: son sous-total, au cent près.

Étape 6 — La preuve : les tests et le témoin

Les tests génériques sont au Cours 05 (Theorie_05.md, section 2), et la réconciliation en jointure externe complète à la section 3.

Le fichier `_marts.yml` pose quinze tests : l'unicité et la présence de chaque clé de dimension, la liste fermée des six catégories avec son `not_null` à côté, le grain du fait, et les trois relationships du fait vers ses dimensions. Celui du membre est le seul qui compte vraiment ici :

```

- name: cle_membre
  description: "Nulle quand aucune carte n'a été présentée."
  data_tests:
    - relationships:
        arguments:
          to: ref('dim_membre')
          field: cle_membre

```

Comme tous les tests génériques, `relationships` ignore les valeurs nulles : les ventes sans carte passent, et une clé qui pointerait vers un membre inexistant tomberait. Mesuré en retirant le case de l'étape 5 : le test échoue sur 1 626 lignes.

Le témoin indépendant compare le fait à ce que la caisse a calculé de son côté, le sous-total de chaque ticket payé. Les deux chiffres ne viennent pas du même chemin, et c'est ce qui fait du second un témoin :

```

with fait as (

  select ticket_id, sum(montant) as montant

```

```

    from {{ ref('fct_ventes') }}
    group by 1

),

caisse as (

    select ticket_id, sous_total as montant
    from {{ ref('stg_tickets') }}
    where statut = 'PAYE'

)

select
    coalesce(fait.ticket_id, caisse.ticket_id) as ticket_id,
    fait.montant as montant_du_fait,
    caisse.montant as sous_total_de_la_caisse

from fait
full outer join caisse on caisse.ticket_id = fait.ticket_id

where fait.ticket_id is null
    or caisse.ticket_id is null
    or fait.montant <> caisse.montant

```

L'égalité est stricte, sans tolérance, parce que les montants sont décimaux depuis l'étape 2. Le test mord : sans le filtre sur le statut au fait, il échoue sur les 15 tickets annulés.

Tout se lance d'un geste, depuis 02_ae :

```
dbt build
```

Attendu : PASS=28 WARN=0 ERROR=0, soit 4 seeds, 4 vues, 4 tables et 16 tests, en moins de deux secondes d'exécution.

Étape 7 — Documenter, et répondre

La documentation est au Cours 05 (Theorie_05.md, section 9).

`dbt docs generate` écrit le catalogue et le manifest.json dans target/, et `dbt docs serve` ouvre le site, où le graphe va des quatre seeds à l'étoile.

Reste une question d'analyste, les ventes du mois par catégorie. Dans la ligne de commande DuckDB :

```
select p.categorie, sum(f.montant) as ventes
from marts.fct_ventes as f
inner join marts.dim_produit as p on p.cle_produit = f.cle_produit
group by p.categorie
order by p.categorie;
```

Catégorie	Ventes
ACCESSOIRE	627,00 \$
BOISSON_CHAUDE	5 638,00 \$
BOISSON_FROIDE	2 291,25 \$
GRAINS	3 721,50 \$
SANDWICH	2 033,00 \$
VIENNOISERIE	2 407,25 \$

Soit 16 718,00 \$, le total de la caisse. Les six chiffres sont ceux que le générateur a écrits dans `_univers/brulerie/CHIFFRES_ATTENDUS.md`, section 2.3, et le Latte reste le produit le plus vendu, 264 fois dans le mois.

Piège — la jointure interne vers `dim_membre` perd les ventes sans carte. Jointe à la dimension des membres par une jointure interne, la même étoile ne garde que 883 lignes et 5 607,00 \$: les 11 111,00 \$ vendus sans carte disparaissent, sans message. Une dimension dont la clé peut être nulle se joint à gauche.

Récapitulatif

Démarche	Où elle est enseignée	Où ici
le projet, le profil, la macro des schémas	Cours 03, sections 2, 3 et 5	Étape 1
le seed, et pourquoi il s'appelle par <code>ref()</code>	Cours 07, section 8 ; Cours 03, sections 4 et 9	Étape 2.1
les types d'un seed, déclarés au chargement	ce mini-cas	Étape 2.2
le processus, le grain, les dimensions, les mesures	Cours 02, sections 2 à 4	Étape 3
ce qui est une vente, et le grain des taxes	ce mini-cas	Étapes 3.1 et 3.2
les quatre gestes du STAGING, la minimisation	Cours 03, sections 6 et 7 ; Cours 06, section 6	Étape 4

Démarche	Où elle est enseignée	Où ici
les clés fabriquées, dim_date, le fait	Cours 04, sections 3, 4 et 7	Étape 5
la clé facultative, et sa valeur nulle	Cours 04, section 3 ; ce mini-cas	Étape 5
les tests génériques, la réconciliation	Cours 05, sections 2 et 3	Étape 6
la documentation	Cours 05, section 9	Étape 7

Ce que le mini-cas ne fait pas, parce qu'un mois figé n'a ni histoire ni lot suivant :

Ce qui manque	Où le trouver
la couche INTERMEDIATE	Cours 04, section 6 : le fait lit directement le STAGING, rien n'est à préparer
le snapshot et la dimension de type 2	Cours 05, sections 6 à 8 : aucun prix ne change dans le mois
l'incrémental	Cours 07, sections 3 à 5 : aucun lot n'arrive après juin
les deux environnements	Cours 07, section 9
la couche PRESENTATION	Cours 05, section 11

Le même pipeline que Boréal, réduit à un mois et à une étoile. Tout ce qui s'y calcule se recompte contre la caisse, et un ticket s'y suit de tête du fichier jusqu'au fait.

Annexe — L'arborescence et les commandes

```

Projet_Brulerie/
├─ mini_cas.duckdb      la base, hors du dépôt
├─ 02_ae/              le dépôt du mini-cas
│   ├── dbt_project.yml  les seeds dans raw, le staging en vues, l'étoile en tables
│   ├── packages.yml     dbt_utils
│   ├── .gitignore
│   ├── macros/generate_schema_name.sql
│   ├── seeds/           les quatre CSV de graines/, versionnés
│   ├── models/
│   │   ├── staging/     stg_tickets, stg_lignes, stg_produits, stg_membres
│   │   └─ marts/        dim_date, dim_produit, dim_membre, fct_ventes, _marts.yml
└─ tests/assert_reconciliation_ventes.sql

```

Pour tout rejouer de zéro, depuis `Projet_Brulerie/02_ae`, env formations actif :

```
dbt deps
dbt seed --full-refresh
dbt build
```

Le corrigé complet, commentaires compris, est dans `Solution/02_ae/`. Son `README.md` dit comment le rejouer sans rien retaper.